

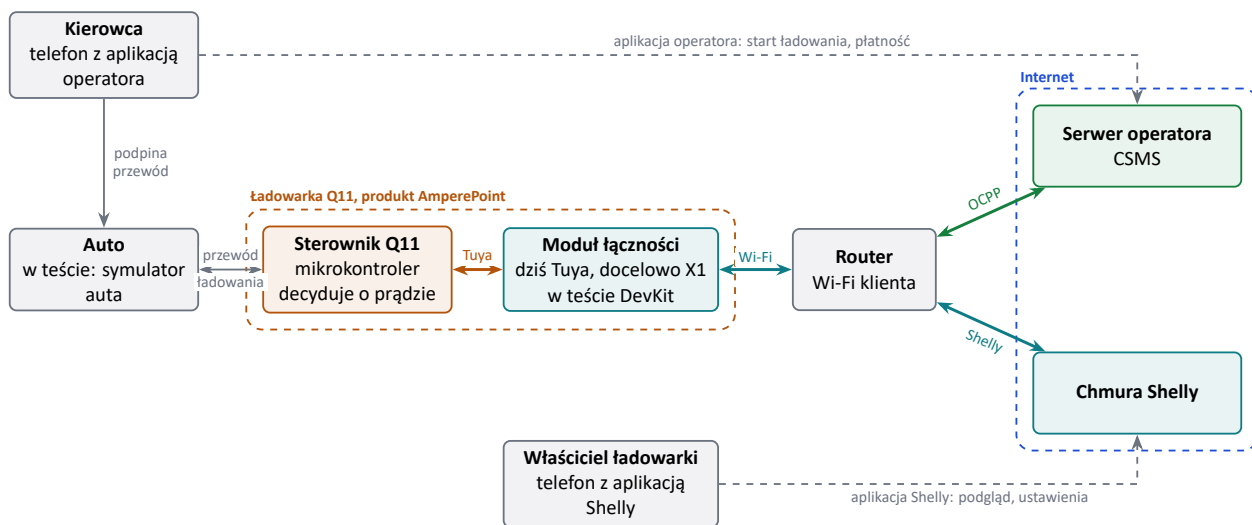
OCPP dla Q11: kto jest kim i cztery sposoby podłączenia

Strony, schematy i porównanie z testem na DevKicie · AMPERE POINT · 27 września 2026 · oprac. Dawid Cekała

W skrócie

- OCPP to język, którym ładowarka rozmawia z serwerem operatora, czyli firmy rozliczającej ładowanie. Ładowarka sama łączy się z tym serwerem i utrzymuje połączenie przez cały czas.
- Q11 nie zna OCPP. Potrzebny jest tłumacz, który z jednej strony mówi OCPP, a z drugiej ustawia pola modułu Shelly. W teście 25.09 tłumaczem był most, czyli program `most_q11.py` na laptopie.
- Cztery rozwiązania różnią się tylko miejscem, w którym działa tłumacz: w systemie modułu X1, w naszym skrypcie w X1, na serwerze AmperePoint albo na małym komputerze u klienta.
- Część między sterownikiem Q11 a modułem, czyli protokół Tuya i nasz skrypt usługi, jest we wszystkich rozwiązaniach taka sama. Działa już dziś.
- Rekomendacja: rozwiązanie 3 od razu, a Shelly zapytać o rozwiązanie 1. Uzasadnienie w rozdziale „Rekomendacja”.

1 Strony: kto jest kim



Auto nie ma dostępu do sieci ładowarki. Rozmawia ze sterownikiem tylko przez przewód ładowania. Operator i Shelly to dwie niezależne drogi.

— język Tuya — język Shelly — OCPP — niezależne od OCPP ■ tu działa tłumacz OCPP

Rysunek 1: Wszystkie strony wokół ładowarki. Droga do operatora (zielona) i droga do Shelly (turkusowa) są od siebie niezależne.

Strona	Czym jest	W teście 25.09
Kierowca	Osoba, która ładuje auto. Ładowanie zaczyna w aplikacji operatora albo kartą.	my, polecenie start w pliku polecenia.txt

Strona	Czym jest	W teście 25.09
Auto	Pobiera prąd. Ze sterownikiem ładowarki rozmawia tylko przez przewód ładowania, sygnałem pilota. Do sieci ładowarki nie ma dostępu.	symulator auta
Sterownik Q11	Mikrokontroler na płytce ładowarki. Włącza i wyłącza prąd, mierzy, pilnuje bezpieczeństwa. Nie ma Wi-Fi.	płytkę Q11 w biurze
Moduł łączności	Mała płytka z Wi-Fi w obudowie Q11. Dziś Tuya WBR3, docelowo Shelly X1.	DevKit, 192.168.0.238
Router klienta	Łączy moduł z internetem. Przepuszcza połączenia wychodzące, blokuje przychodzące.	router biura
Operator	Firma, która prowadzi ładowanie jako usługę: wydaje karty, nalicza opłaty, widzi stan ładowarek, zdalnie uruchamia i ogranicza ładowanie. Po angielsku Charge Point Operator.	rolę grał serwer testowy
Serwer operatora, CSMS	Program operatora, z którym łączą się ładowarki. Nazwa od Charging Station Management System. Operator może mieć własny albo wynająć gotowy, np. Monta lub Virta.	csms_test.py na laptopie, port 9000
Tłumacz OCPP	Program, który mówi OCPP do operatora i ustawia pola modułu. Nazywany też mostem albo bramką.	most_q11.py na laptopie
Chmura i aplikacja Shelly	Serwery Shelly i aplikacja w telefonie właściciela. Podgląd i ustawienia, bez rozliczeń. Działa niezależnie od OCPP.	wyłączona
Właściciel ładowarki	Ten, kto kupił Q11. W domu to zwykle ta sama osoba co kierowca, na parkingu firma.	biuro
AmperePoint	Producent Q11. Ustala produkt w portalu Shelly X i pisze skrypt usługi. W rozwiązaniu 3 prowadzi też serwer z tłumaczem.	my
Shelly	Producent modułu X1 i jego systemu. Tylko Shelly może zmienić system w module.	brak udziału
Instalator	Montuje ładowarkę u klienta i wpisuje ustawienia, w tym adres operatora.	my

1.1 Kiedy w ogóle potrzebny jest operator

Operator jest potrzebny wtedy, gdy za prąd płaci ktoś inny niż kierowca albo gdy ktoś musi ładowanie rozliczyć. Przykłady:

- firma z samochodami służbowymi, która zwraca pracownikom koszt ładowania w domu,
- hotel lub parking, który sprzedaje ładowanie gościom,
- sieć publicznych ładowarek.

Przykład liczbowy: pracownik ładuje służbowe auto w domu, 300 kWh w miesiącu. Przy cenie 1,20 zł za kWh firma zwraca mu 360 zł. Żeby to policzyć, operator dostaje z ładowarki stan licznika energii na początku i na końcu każdej sesji.

W domu, gdzie kierowca płaci własny rachunek za prąd, OCPP zwykle nie jest potrzebne. Wystarczy aplikacja.

2 OCPP w jednym akapicie

Ładowarka jest klientem, serwer operatora jest serwerem. Ładowarka otwiera jedno stałe połączenie pod adres operatora z dopisanym swoim identyfikatorem, np. `wss://csms.przyklad-operatora.pl/ocpp/Q11-543204516334`. Połączenie zaczyna zawsze ładowarka, bo stoi za routerem klienta, a router nie wpuszcza połączeń z internetu. Gdy połączenie już jest, działa w obie strony.

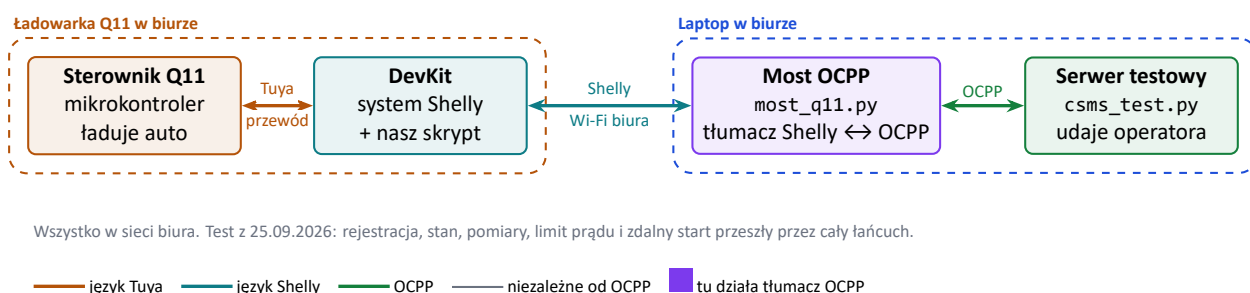
To stałe połączenie to WebSocket. Działa jak rozmowa telefoniczna, która trwa cały czas: każda strona może się odezwać w dowolnej chwili. Zwykle zapytanie HTTP działa jak list z odpowiedzią: zaczyna zawsze pytający. OCPP potrzebuje telefonu, bo operator musi móc w każdej chwili powiedzieć „ogranicz do 10 A” albo „zacznij ładowanie”.

3 Część wspólna wszystkich rozwiązań

Sterownik Q11 i moduł rozmawiają protokołem Tuya przez przewód. Nasz skrypt usługi w module zamienia punkty danych Tuya na pola Shelly i z powrotem. Na przykład limit prądu to w Q11 punkt danych 4, a w module pole limitu prądu.

Ta część jest sprawdzona. Limit 10 A i 12 A ustawiony przez MQTT i przez OCPP Q11 potwierdziła swoim mel-dunkiem. Każde z czterech rozwiązań korzysta z niej bez zmian. Rozwiązania różnią się tylko tym, kto ustawia pola i skąd.

4 Punkt odniesienia: test na DevKicie



Wszystko w sieci biura. Test z 25.09.2026: rejestracja, stan, pomiary, limit prądu i zdalny start przeszły przez cały łańcuch.

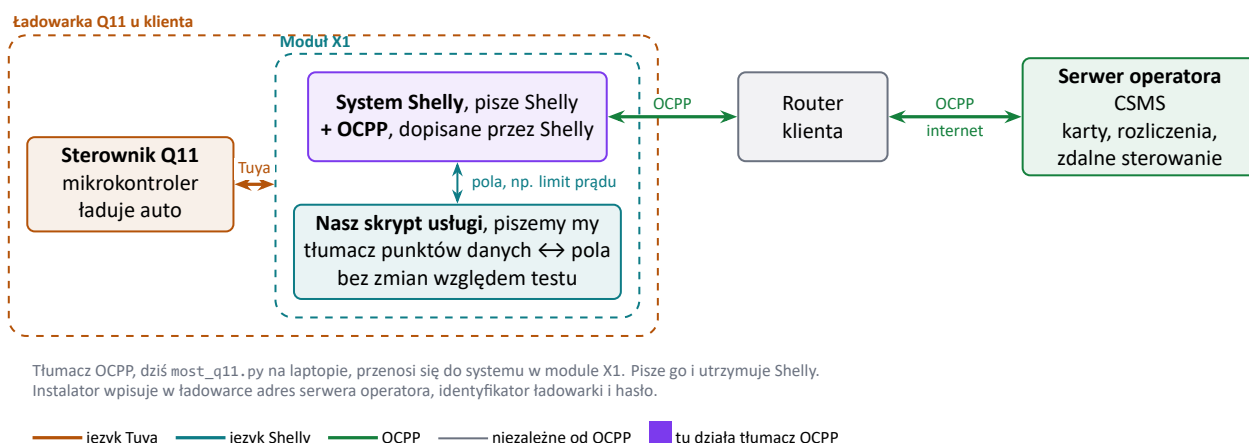
Rysunek 2: Test z 25.09.2026. Tłumacz i serwer testowy działały na laptopie w biurze.

Przykład: operator ogranicza ładowanie do 10 A.

1. Serwer testowy wysyła polecenie OCPP `SetChargingProfile` z limitem 6900 W.
2. Most na laptopie przelicza waty na ampery: 6900 W podzielone przez 690 daje 10 A. Liczba 690 to trzy fazy razy 230 V.
3. Most wysyła do DevKitu przez Wi-Fi polecenie w języku Shelly: ustaw pole limitu prądu na 10.
4. Nasz skrypt w DevKicie układa ramkę Tuya `55 AA 00 06 00 08 04 02 00 04 00 00 00 0A 21` i wysyła ją do sterownika.
5. Sterownik najpierw odsyła starą wartość, a po około sekundzie nową, 10 A.

U klienta tak zostać nie może. Laptop musiałby działać cały czas i stać w tej samej sieci co ładowarka. Każde z rozwiązań poniżej usuwa laptop w inny sposób.

5 Rozwiązanie 1: OCPP w systemie modułu X1, pisze Shelly



Rysunek 3: Rozwiązanie 1. Tłumacz przenosi się z laptopa do systemu modułu X1.

5.1 Jak to działa względem testu

W teście tłumaczem był most na laptopie. W rozwiązaniu 1 Shelly dopisuje taki sam tłumacz do systemu modułu. Laptop znika, a moduł sam łączy się z operatorem przez router klienta.

Krok	Test na DevKicie	Rozwiązanie 1
Kto otwiera połączenie z operatorem	most na laptopie	moduł X1
Skąd bierze adres operatora	wpisany w programie mostu	z ustawień modułu
Kto przelicza 6900 W na 10 A	most	kod OCPP w systemie X1
Kto ustawia pole limitu prądu	most, poleceniem przez Wi-Fi	kod OCPP w systemie X1, wewnątrz modułu
Kto układa ramkę Tuya	nasz skrypt	nasz skrypt, bez zmian
Kto zgłasza operatorowi stan „ładuje”	most, po odczycie pola stanu	kod OCPP w systemie X1

5.2 Gdzie wpisuje się adres operatora

Tak by to prawdopodobnie wyglądało. Szczegóły ustala Shelly.

- **W portalu Shelly X, raz dla wszystkich Q11:** włączenie OCPP i opis, które pole jest limitem prądu, stanem, licznikiem energii. Dziś tę pracę wykonuje most w swoich funkcjach.
- **W każdej ładowarce osobno:** adres serwera operatora, identyfikator ładowarki i hasło. Każdy klient może mieć innego operatora, więc tego nie da się wpisać w portalu raz dla wszystkich. Wpisuje instalator na stronie modułu albo w aplikacji.

5.3 Co musi zrobić Shelly i czy starczy pamięci

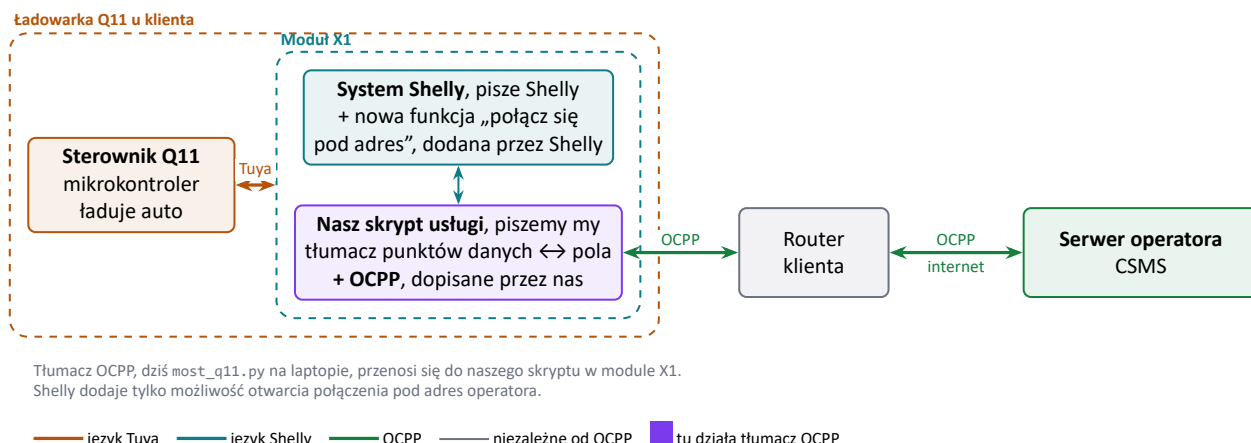
Shelly musi dopisać do systemu klienta OCPP 1.6J z szyfrowaniem TLS i hasłem, ustawienia dla instalatora i powiązanie z polami. My dostarczamy opis pól.

Zasób X1	Wolne dziś	Potrzeba OCPP z TLS	Ocena
Pamięć programu	689 kB	120 do 200 kB	✓ mieści się, zapas ponad trzykrotny
Pamięć robocza	84 do 97 KB	OCPP 12 do 22 KB, TLS 40 do 50 KB przy łączeniu	🟡 ciasno, w szczycie zostaje 12 do 45 KB

Wolne miejsce zmierzaliśmy na DevKicie, który ma ten sam układ co X1. Potrzeby OCPP i TLS pochodzą z danych o bibliotekach, źródła są na końcu. Shelly zna swoją pamięć dokładniej i to on oceni, czy się zmieści.

- ✓ Bez dodatkowego sprzętu i bez naszego serwera. Każda Q11 działa sama.
- ✓ Najprostsze dla klienta i instalatora.
- ✗ Zależymy od Shelly: czy to robi, kiedy i za ile.
- ✗ Każdą poprawkę w OCPP robi Shelly w kolejnym wydaniu systemu.

6 Rozwiązanie 2: OCPP w naszym skrypcie w module X1



Rysunek 4: Rozwiązanie 2. Tłumacz przenosi się do naszego skryptu w module X1. Shelly dodaje tylko funkcję połączenia.

6.1 W jakim module

W tym samym module X1, który siedzi w Q11. Oprogramowanie modułu ma dwie warstwy:

- **System**, który pisze Shelly: Wi-Fi, szyfrowanie, połączenia, obsługa łącza z ładowarką, strona WWW, MQTT. Tej warstwy nie możemy zmienić.
- **Skrypt usługi**, który piszemy my i wgrywamy przez portal. To nasz plik `script.svc.ts`, dziś 285 wierszy. Zamienia punkty danych Tuya na pola.

Działa to jak telefon: system pisze producent telefonu, aplikacje piszą inni. Aplikacja może tylko to, na co pozwala jej system.

6.2 Czego dziś brakuje

Nasz skrypt może dziś rozmawiać ze sterownikiem, ustawiać pola i odmierzać czas. W spisie funkcji modułu z 25.09 są też zapytania HTTP. Nie ma funkcji, która otwiera stałe połączenie WebSocket pod wybrany adres. HTTP nie wystarczy. W HTTP zawsze pyta moduł, więc operator nie może sam wysłać polecenia. Moduł musiałby co chwilę pytać, czy są nowe polecenia. Przy pytaniu co 5 sekund to 17 280 zapytań na dobę z jednej ładowarki. Poza tym OCPP 1.6J z definicji działa przez WebSocket, więc serwer operatora zapytań HTTP i tak by nie przyjął.

6.3 Jak to działa względem testu

Shelly dodaje do systemu jedną ogólną funkcję dla skryptów: otwórz WebSocket pod adres, wyślij, odbierz. To dużo mniej pracy niż cały klient OCPP i przyda się też innym klientom Shelly. Logikę mostu, dziś 363 wiersze w Pythonie, przepisujemy na JavaScript do naszego skryptu.

Krok	Test na DevKicie	Rozwiązanie 2
Kto otwiera połączenie z operatorem	most na laptopie	nasz skrypt, funkcją od Shelly
Kto przelicza 6900 W na 10 A	most	nasz skrypt

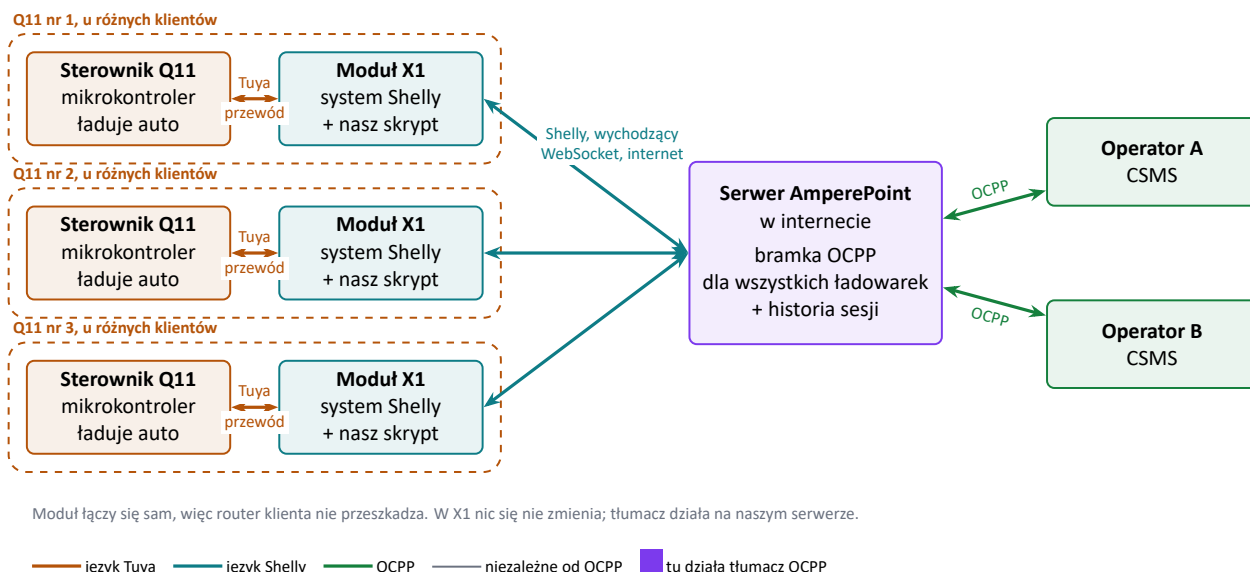
Krok	Test na DevKicie	Rozwiązanie 2
Kto układa ramkę Tuya	nasz skrypt	nasz skrypt, od razu, bez przechodzenia przez pole
Jak poprawić błąd w OCPP	zmienić plik na laptopie	wgrać nowy skrypt przez portal; nie kasuje to ustawień Wi-Fi

6.4 Czy starczy pamięci

🟡 To najbardziej niepewne z czterech rozwiązań. Dziś wolne jest 84 do 97 KB pamięci roboczej. Połączenie TLS zabiera przy łączeniu 40 do 50 KB, więc na skrypt z OCPP zostaje 34 do 57 KB. Skrypt działa w silniku JavaScript, a taki kod zużywa więcej pamięci niż ten sam kod wbudowany w system. Każda wiadomość OCPP to tekst, który skrypt musi rozłożyć w pamięci na części. Czy się zmieści, pokaże dopiero pomiar na module z tą funkcją.

- ✓ Bez dodatkowego sprzętu. OCPP w naszych rękach, poprawki wgrywamy przez portal.
- ✓ Od Shelly potrzebna jest tylko jedna ogólna funkcja.
- 🟡 Pamięć niepewna.
- ✗ Nadal zależymy od Shelly, choć w mniejszym stopniu.
- ✗ Dużo naszej pracy: przepisanie mostu i testy OCPP w JavaScript.

7 Rozwiązanie 3: tłumacz na serwerze AmperePoint



Rysunek 5: Rozwiązanie 3. Moduły łączą się z serwerem AmperePoint, a on rozmawia z operatorami.

7.1 Jak to działa względem testu

To nasz most z testu, tylko przeniesiony z laptopa na serwer w internecie i obsługujący wiele ładowarek naraz.

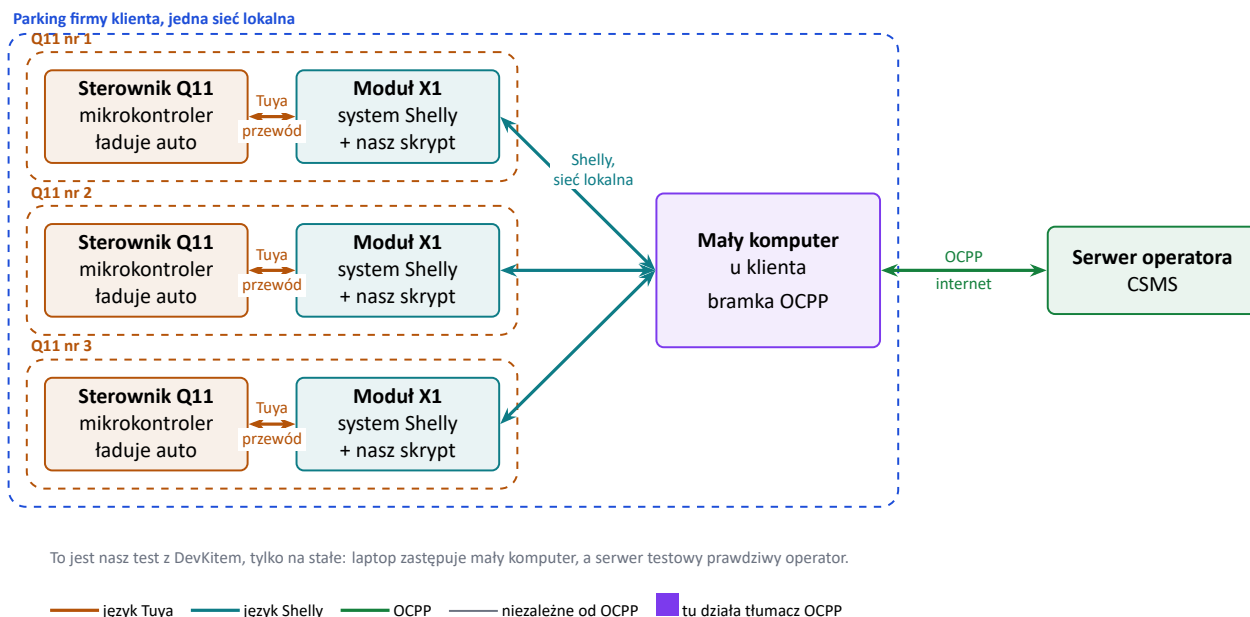
- Każdy moduł X1 sam łączy się z serwerem AmperePoint i rozmawia z nim językiem Shelly, tak jak w teście DevKit z mostem. Połączenie jest wychodzące, więc router klienta go przepuszcza.
- Moduł ma to już w systemie. W ustawieniach DevKitu z 25.09 jest pole wychodzącego WebSocketu: wyłączone, adres pusty. Nie próbowaliśmy go jeszcze włączyć. Druga droga, sprawdzona w teście, to MQTT do brokera, w internecie z TLS i hasłem.
- Na serwerze dla każdej Q11 działa osobny tłumacz. łączy się z operatorem tej ładowarki pod jej identyfikatorem, np. Q11-543204516334. Operator widzi zwykłą ładowarkę OCPP i nie musi wiedzieć o pośredniku.
- Różne Q11 mogą mieć różnych operatorów. Serwer trzyma dla każdej adres, identyfikator i hasło.

Przykład 10 A: operator wysyła SetChargingProfile z limitem 6900 W do serwera AmperePoint. Tłumacz przelicza to na 10 A i wysyła polecenie Shelly przez połączenie, które moduł wcześniej otworzył. Dalej jest jak w teście: skrypt, ramka Tuya, sterownik.

Obciążenie serwera, przykład dla 500 ładowarek: znak życia co 60 s i pomiary co 30 s dają 3 wiadomości na minutę z jednej ładowarki, razem 25 na sekundę. Mniej więcej tyle samo przychodzi od modułów. To niewielki ruch nawet dla małego serwera.

- ✓ Nic nie trzeba od Shelly. Kod tłumacza już jest i działa w teście.
- ✓ W X1 nic się nie zmienia, więc pamięć modułu nie jest zagrożona.
- ✓ Poprawki robimy na serwerze, bez dotykania ładowarek. Na serwerze można też trzymać historię sesji dla klientów.
- ✗ Musimy utrzymywać serwer całą dobę: koszt, zabezpieczenia, odpowiedzialność za dane klientów.
- ✗ Gdy nasz serwer przestanie działać, wszystkie Q11 tracą połączenie z operatorami. Sterownik Q11 nadal ładuje sam, a aplikacja Shelly działa dalej, bo idzie inną drogą.

8 Rozwiązanie 4: tłumacz na komputerze u klienta



Rysunek 6: Rozwiązanie 4. Mały komputer w sieci klienta obsługuje jego ładowarki.

To jest nasz test, tylko na stałe. Laptop zastępuje mały komputer, np. Raspberry Pi, który stoi w sieci klienta i działa całą dobę. Program jest ten sam, `most_q11.py`, rozszerzony o obsługę kilku ładowarek.

Pasuje do klienta z kilkoma lub kilkunastoma Q11 w jednym miejscu, np. na parkingu firmy. Komputer może wtedy także dzielić moc przyłącza między ładowarki. Przykład: trzy Q11 po 16 A to razem 48 A na fazę, a przyłącze ma 40 A. Komputer widzi wszystkie trzy i ustawia im limity tak, by suma nie przekroczyła 40 A, również bez internetu.

- ✓ Nic nie trzeba od Shelly. Kod jest sprawdzony w teście.
- ✓ Nie zależy od naszego serwera.
- ✗ Dodatkowe urządzenie u każdego klienta: zakup, montaż, aktualizacje.
- ✗ Przy jednej ładowarce w domu nieopłacalne.

9 Porównanie

Cecha	1. W systemie X1	2. W naszym skrypcie	3. Serwer AmperePoint	4. Komputer u klienta
Gdzie działa tłumacz	moduł X1	moduł X1	nasz serwer	sieć klienta
Kto pisze tłumacza	Shelly	my	my	my
Co musi zrobić Shelly	cały klient OCPP	funkcja WebSocket dla skryptu	nic	nic
Dodatkowy sprzęt	✓ brak	✓ brak	🟡 jeden nasz serwer	✗ komputer u każdego klienta
Pamięć X1	🟡 ciasno	🟡 niepewne	✓ bez zmian	✓ bez zmian
Działa, gdy padnie nasz serwer	✓	✓	✗	✓
Kiedy może ruszyć	gdy zrobi to Shelly	gdy Shelly doda funkcję, plus nasza praca	od razu	od razu
Kto wydaje poprawki OCPP	Shelly	my, przez portal	my, na serwerze	my, na każdym komputerze
Co już sprawdzone	nic	nic	🟡 logika tłumacza	✓ cały łańcuch, to był nasz test

10 Rekomendacja

Rozwiązanie 3 od razu, a Shelly zapytać o rozwiązanie 1.

- Tłumacz z rozwiązania 3 już działa. W teście 25.09 przeszły rejestracja, stan, pomiary, zdalny start i limit prądu 10 A i 12 A, który Q11 potwierdziła. Trzeba go przenieść na serwer i dodać obsługę wielu ładowarek.
- Rozwiązanie 3 nie zmienia nic w module. Pamięć robocza X1, która w rozwiązaniach 1 i 2 jest ciasna, zostaje nietknięta.
- Rozwiązanie 1 jest najwygodniejsze dla klienta, ale zależy od Shelly. Jeśli Shelly je zrobi, Q11 można przełączyć z rozwiązania 3 na 1 zmianą ustawień w module. Operator nie zauważy różnicy, bo identyfikator ładowarki zostaje ten sam.
- Rozwiązanie 2 zostaje planem zapasowym, gdyby Shelly odmówiło rozwiązania 1.
- Rozwiązanie 4 ma sens tylko u klienta z wieloma ładowarkami w jednym miejscu.

10.1 Pytania do Shelly

1. Czy dodacie klienta OCPP 1.6J z TLS do systemu Shelly X? W jakim terminie i za ile?
2. Jeśli nie: czy skrypt usługi może dostać funkcję klienta WebSocket z TLS?
3. Ile pamięci roboczej zostaje dla skryptu usługi na X1 i czy starczy jej obok połączenia TLS?
4. Czy wychodzący WebSocket z ustawień modułu działa na X1 i jak serwer rozpoznaje, który moduł się połączył?

11 Czego jeszcze nie sprawdziliśmy

- Pełnej sesji ładowania z symulatorem auta, który pobiera prąd.

- Licznika energii. Punkt danych 1 podaje dziś 0, a operator potrzebuje stanu licznika na początku i końcu sesji do rozliczenia.
- Wychodzącego WebSocketu modułu do serwera w internecie.
- Połączeń z TLS i hasłem. Test był bez szyfrowania, w sieci biura.

Słowniczek

Pojęcie	Znaczenie
CSMS	serwer operatora, z którym łączą się ładowarki
OCPP 1.6J	język rozmowy ładowarki z serwerem operatora; wiadomości tekstowe JSON przez WebSocket
WebSocket	stałe połączenie, w którym obie strony mogą się odezwać w każdej chwili
HTTP	pojedyncze pytanie i odpowiedź; zaczyna zawsze pytający
TLS	szyfrowanie połączenia; adres zaczyna się wtedy od wss:// zamiast ws://
Tłumacz, most, bramka	program, który po jednej stronie mówi OCPP, a po drugiej językiem Shelly
Język Shelly	polecenia modułu Shelly, np. „ustaw pole limitu prądu na 10”
Skrypt usługi	nasz program w module, wgrywany przez portal Shelly X
Pole	wartość w module Shelly, np. limit prądu, którą można odczytać i ustawić
Punkt danych	wartość w protokole Tuya, np. punkt 4 to w Q11 limit prądu

Źródła

- Pomiary DevKitu z 25.09.2026: spis funkcji i ustawień w DevKit\logi\inwentarz_2026-09-25_1116\, kopia pamięci flash, analiza GAP v3, rozdział 4.
- MicroOcpp, klient OCPP dla mikrokontrolerów
- Open Charge Alliance: OCPP dla urządzeń o małych zasobach
- Open Charge Alliance: minimalne zasoby dla OCPP 2.x, PDF
- Espressif FAQ: pamięć potrzebna mbedTLS
- Espressif: zużycie pamięci RAM w ESP-IDF
- Shelly X: moduły